

COMP1324 Algorithmics

Dynamic Programming

Hikmat Farhat

Introduction

- The basic idea of dynamic programming is to solve a given problem by combining solutions to smaller **subproblems**
- Dynamic Programming applies when a problem has
 - 1 **Optimal substructure**: recurrence relation between the optimal solutions to the problem and its subproblems
 - 2 **Overlapping subproblems**, otherwise it is divide-and-conquer
- Since subproblems overlap we end up computing same subproblems multiple times
- This can be solved by **memoization** or **bottom-up** solutions
- Finding the recurrence is the most difficult part

Need for memoization or bottom-up

- Computing Fibonacci numbers illustrates the need for memoization/bottom-up
- In this case we **already have the recurrence** which is usually the most difficult part of DP
- Recall that

$$F(n) = F(n-1) + F(n-2)$$

with $F(0) = 0, F(1) = 1$

Time Complexity

- Implementing the recurrence directly as shown below leads to an inefficient solution

$FB(n)$

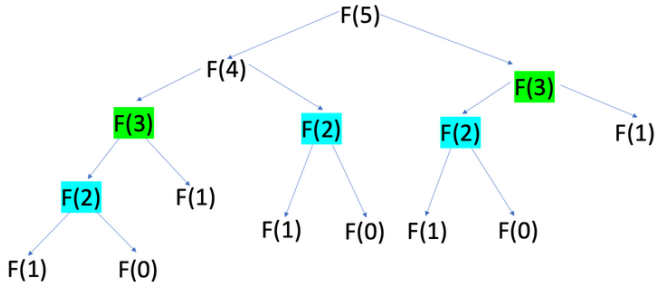
if $n \leq 1$ **then**
 return n

return $FB(n - 1) + FB(n - 2)$

- Knowing that for large n , $F(n) = \Theta(\frac{\phi^n}{\sqrt{5}})$ where $\phi = \frac{1+\sqrt{5}}{2}$ is the golden ratio
- What is the time complexity of the recursive solution?

Time complexity

- The inefficiency is due to the repeated computation of some values
- the recursion tree for $FB(5)$ is shown below where $FB(2)$ is computed 3 times and $FB(3)$ is computed twice.



Memoization

- Store the value of each subproblem to avoid recomputing the same subproblem multiple times.
- Time/space complexity is $\Theta(n)$

Algorithm 1: Memoized Fibonacci

create global array *fibs* of size $n + 1$

for $i = 0$ **to** n **do**

 | $fibs[i] \leftarrow -1$

$fibs[0], fibs[1] \leftarrow 0, 1$

function $FB(n)$

 | **if** $fibs[n] \neq -1$ **then**

 | **return** $fibs[n]$

 | **else**

 | $fibs[n] \leftarrow fibs[n - 1] + fibs[n - 2]$

 | **return** $fibs[n]$

Bottom up

Algorithm 2: Bottom up Fibonacci

```
function FB( $n$ )
```

$$fibs[0], fibs[1] \leftarrow 0, 1$$

for $i = 2$ to n do

$$| \quad fibs[i] \leftarrow fibs[i-1] + fibs[i-2]$$

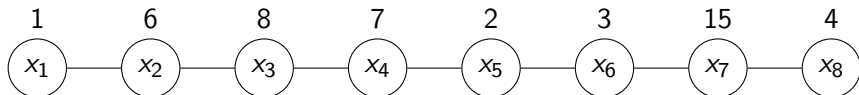
```
return fibs[n]
```

Independent Set (Linear)

- Given a graph $G = (V, E)$ an independent set is a subset of vertices $S \subseteq V$ such that for all $x, y \in S$, $(x, y) \notin E$.
- Usually we want $|S|$ maximum. This problem is hard to solve in the general case (NP hard)
- In this lecture we consider a special case of the independent set: when vertices form a chain.
- The solution for this particular case is trivial: select every other node.
- We make things interesting by adding a weight to each node and ask : what is the independent set which has the largest total weight?

Example

- An example of the (linear) independent set problem with weights is shown below.
- Clearly the solution in this case is $\{x_2, x_4, x_7\}$ for a total value of 28.
- What value is obtained with the simple greedy strategy?



Dynamic Programming Solution

- Given a set of nodes $\{x_1, \dots, x_n\}$ with associated weights $\{w_1, \dots, w_n\}$ we need to find an independent set S such that $\sum_{x \in S} w_x$ is maximum
- Let $\mathcal{O}$ be the optimal solution (which we don't know how to compute yet) with value $opt(n)$.
- The dependence on n comes from the fact that we will express this value in terms of smaller subproblems.
- Consider the last node x_n . Either x_n contributed to $opt(n)$ (i.e. $x_n \in \mathcal{O}$) or not.

- There are two cases to consider
 - ① Either $x_n \in \mathcal{O}$, implies that $x_{n-1} \notin \mathcal{O}$. If, somehow, we can compute the optimal value for $\{x_1, \dots, x_{n-2}\}$ then we add to it w_n to obtain the optimal value
 - ② Or $x_n \notin \mathcal{O}$, implies that the optimal value is the same as the one obtained for $\{x_1, \dots, x_{n-1}\}$
- Which case should we choose?

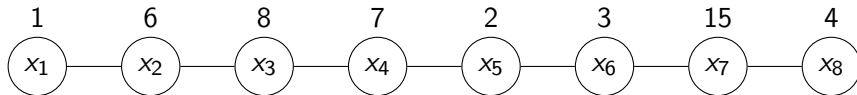
Dynamic Programming Solution

- From the analysis above we conclude that

$$opt(n) = \max \begin{cases} w_n + opt(n-2) \\ opt(n-1) \end{cases}$$

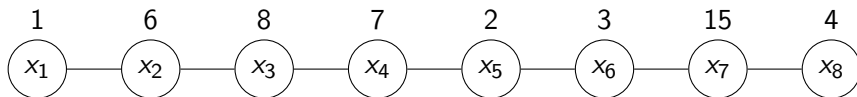
- Applying the above recursive formula to the previous example while noting that $opt[0] = 0$, $opt[1] = 2$ (only x_1 is included) we get

Example (Top down)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

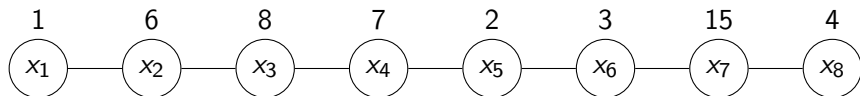
Example (Top down)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

Example (Top down)

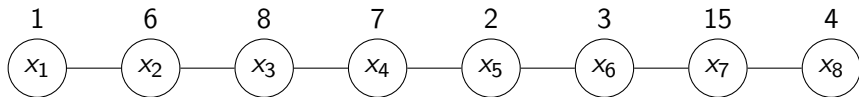


$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

Example (Top down)



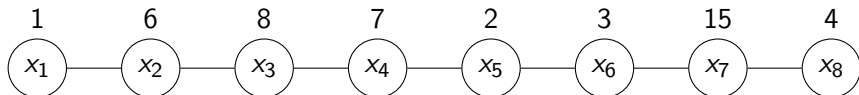
$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

$$opt(5) = \max \begin{cases} 2 + opt(3) \\ opt(4) \end{cases}$$

Example (Top down)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

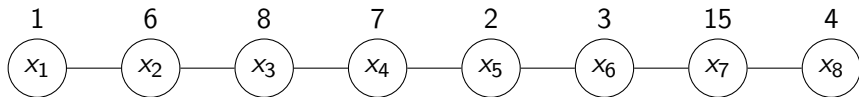
$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

$$opt(4) = \max \begin{cases} 7 + opt(2) \\ opt(3) \end{cases}$$

$$opt(5) = \max \begin{cases} 2 + opt(3) \\ opt(4) \end{cases}$$

Example (Top down)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

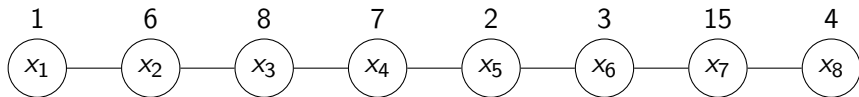
$$opt(4) = \max \begin{cases} 7 + opt(2) \\ opt(3) \end{cases}$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

$$opt(3) = \max \begin{cases} 8 + opt(1) \\ opt(2) \end{cases}$$

$$opt(5) = \max \begin{cases} 2 + opt(3) \\ opt(4) \end{cases}$$

Example (Top down)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

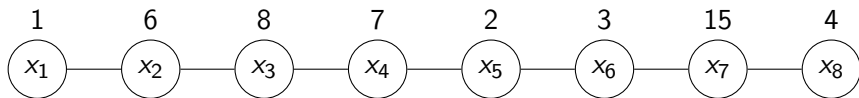
$$opt(5) = \max \begin{cases} 2 + opt(3) \\ opt(4) \end{cases}$$

$$opt(4) = \max \begin{cases} 7 + opt(2) \\ opt(3) \end{cases}$$

$$opt(3) = \max \begin{cases} 8 + opt(1) \\ opt(2) \end{cases}$$

$$opt(2) = \max \begin{cases} 6 + opt(0) \\ opt(1) \end{cases}$$

Example (Top down)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = \max \begin{cases} 7 + opt(2) \\ opt(3) \end{cases}$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(3) = \max \begin{cases} 8 + opt(1) \\ opt(2) \end{cases}$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

$$opt(2) = \max \begin{cases} 6 + opt(0) \\ opt(1) \end{cases}$$

$$opt(5) = \max \begin{cases} 2 + opt(3) \\ opt(4) \end{cases}$$

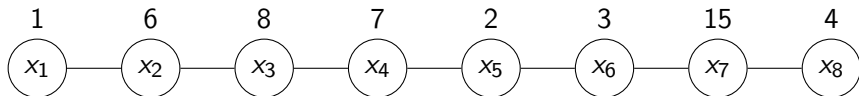
$$opt(0) = 0$$

$$opt(1) = 1$$

Looking for an efficient solution

- It is clear that the above recursive solution is inefficient since we are computing the same subproblems multiple times.
- We can either use memoization, i.e. store the value of each subproblem once it is computed or
- Or implement the recursion using a bottom-up approach, i.e. start from the smallest subproblem and build the solution iteratively.
- Either way each subproblem is computed only once

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = \max \begin{cases} 7 + opt(2) \\ opt(3) \end{cases}$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(3) = \max \begin{cases} 8 + opt(1) \\ opt(2) \end{cases}$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

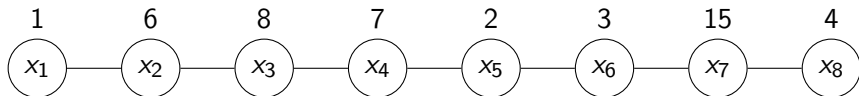
$$opt(2) = \max \begin{cases} 6 + opt(0) \\ opt(1) \end{cases}$$

$$opt(5) = \max \begin{cases} 2 + opt(3) \\ opt(4) \end{cases}$$

$$opt(0) = 0$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = \max \begin{cases} 7 + opt(2) \\ opt(3) \end{cases}$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(3) = \max \begin{cases} 8 + opt(1) \\ opt(2) \end{cases}$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

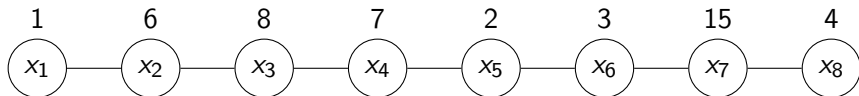
$$opt(2) = \max \begin{cases} 6 \\ 1 \end{cases}$$

$$opt(5) = \max \begin{cases} 2 + opt(3) \\ opt(4) \end{cases}$$

$$opt(0) = 0$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = \max \begin{cases} 7 + opt(2) \\ opt(3) \end{cases}$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(3) = \max \begin{cases} 8 + opt(1) \\ opt(2) \end{cases}$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

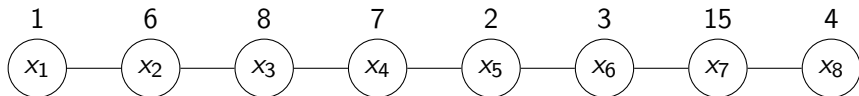
$$opt(2) = 6$$

$$opt(5) = \max \begin{cases} 2 + opt(3) \\ opt(4) \end{cases}$$

$$opt(0) = 0$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = \max \begin{cases} 7 + opt(2) \\ opt(3) \end{cases}$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(3) = \max \begin{cases} 9 \\ 6 \end{cases}$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

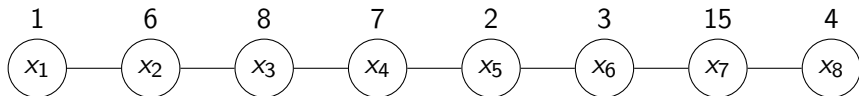
$$opt(2) = 6$$

$$opt(5) = \max \begin{cases} 2 + opt(3) \\ opt(4) \end{cases}$$

$$opt(0) = 0$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = \max \begin{cases} 7 + opt(2) \\ opt(3) \end{cases}$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(3) = 9$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

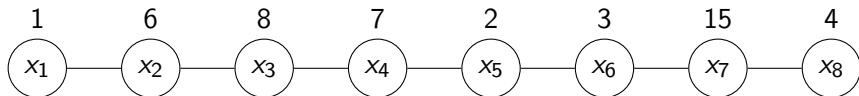
$$opt(2) = 6$$

$$opt(5) = \max \begin{cases} 2 + opt(3) \\ opt(4) \end{cases}$$

$$opt(0) = 0$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = \max \begin{cases} 13 \\ 9 \end{cases}$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(3) = 9$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

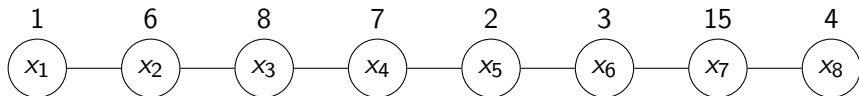
$$opt(2) = 6$$

$$opt(5) = \max \begin{cases} 2 + opt(3) \\ opt(4) \end{cases}$$

$$opt(0) = 0$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = 13$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(3) = 9$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

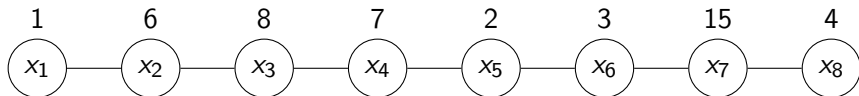
$$opt(2) = 6$$

$$opt(5) = \max \begin{cases} 2 + opt(3) \\ opt(4) \end{cases}$$

$$opt(0) = 0$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = 13$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(3) = 9$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

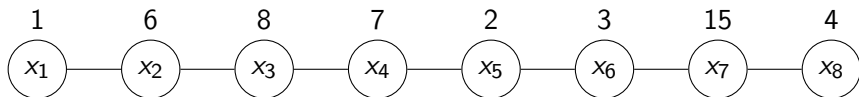
$$opt(2) = 6$$

$$opt(5) = \max \begin{cases} 11 \\ 13 \end{cases}$$

$$opt(0) = 0$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = 13$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(3) = 9$$

$$opt(6) = \max \begin{cases} 3 + opt(4) \\ opt(5) \end{cases}$$

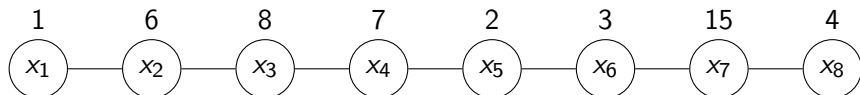
$$opt(2) = 6$$

$$opt(5) = 13$$

$$opt(0) = 0$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = 13$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(3) = 9$$

$$opt(6) = \max \begin{cases} 16 \\ 13 \end{cases}$$

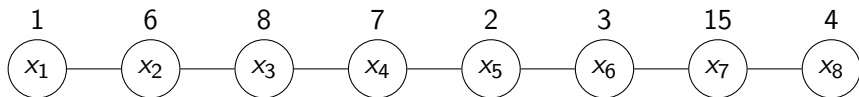
$$opt(2) = 6$$

$$opt(5) = 13$$

$$opt(0) = 0$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = 13$$

$$opt(7) = \max \begin{cases} 15 + opt(5) \\ opt(6) \end{cases}$$

$$opt(3) = 9$$

$$opt(2) = 6$$

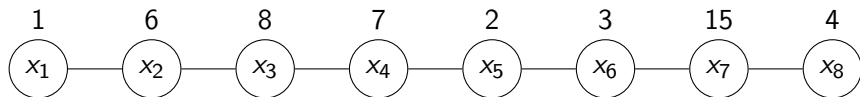
$$opt(6) = 16$$

$$opt(0) = 0$$

$$opt(5) = 13$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = 13$$

$$opt(3) = 9$$

$$opt(7) = \max \begin{cases} 28 \\ 16 \end{cases}$$

$$opt(2) = 6$$

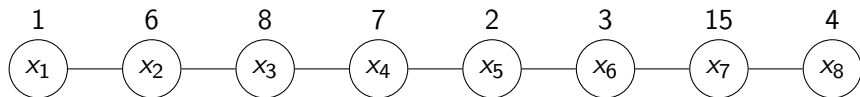
$$opt(6) = 16$$

$$opt(0) = 0$$

$$opt(5) = 13$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 4 + opt(6) \\ opt(7) \end{cases}$$

$$opt(4) = 13$$

$$opt(3) = 9$$

$$opt(7) = 28$$

$$opt(2) = 6$$

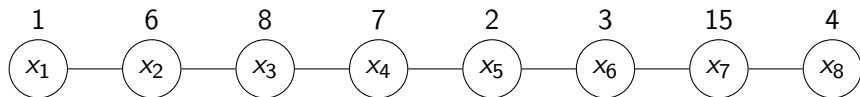
$$opt(6) = 16$$

$$opt(0) = 0$$

$$opt(5) = 13$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(8) = \max \begin{cases} 20 \\ 28 \end{cases}$$

$$opt(7) = 28$$

$$opt(6) = 16$$

$$opt(5) = 13$$

$$opt(4) = 13$$

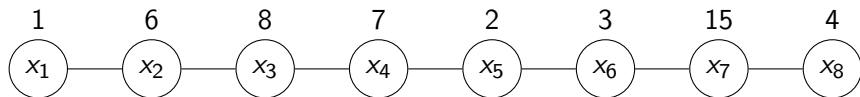
$$opt(3) = 9$$

$$opt(2) = 6$$

$$opt(0) = 0$$

$$opt(1) = 1$$

Example (Bottom up)



$$opt(4) = 13$$

$$opt(8) = 28$$

$$opt(3) = 9$$

$$opt(7) = 28$$

$$opt(2) = 6$$

$$opt(6) = 16$$

$$opt(0) = 0$$

$$opt(5) = 13$$

$$opt(1) = 1$$

Finding the solution

- We have compute the **optimal value**. What if we want the **actual solution**?

Finding the solution

- We have compute the **optimal value**. What if we want the **actual solution**? walk backwards.

Finding the solution

- We have compute the **optimal value**. What if we want the **actual solution**? walk backwards.
- Recall the recurrence

$$opt(n) = \max \begin{cases} w_n + opt(n-2) \\ opt(n-1) \end{cases}$$

- Since $opt(8) = 28$ and $opt(7) = 28$ then x_8 was **not** selected in the solution
- Since $opt(7) = 28$ and $opt(16) = 16$ then x_7 **was** selected in the solution, it follows that x_6 cannot be in the solution. etc...

Pseudocode

Algorithm 3: DP solution for independent set

input: array of values of nodes w

Result: Returns array opt the optimal values for independent set

function $opt_val(w)$

$opt[0], opt[1] \leftarrow 0, w[1]$

for $i = 2$ **to** n **do**

$opt[i] = \max(opt[i - 1], w[i] + opt[i - 2])$

return opt

Pseudocode

Algorithm 4: Optimal solution for independent set

input: array of optimal values opt

Result: Builds optimal solution for independent set

function $opt\text{-}sol(opt)$

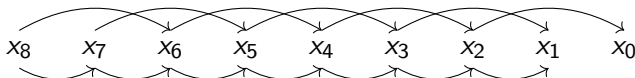
```

     $i, S \leftarrow n, \emptyset$ 
    while  $i > 0$  do
        if  $opt[i] > opt[i - 1]$  then
             $S = S \cup \{i\}$ 
             $i \leftarrow i - 2$ 
        else
             $i \leftarrow i - 1$ 
    return  $S$ 

```

Subproblem graph

- In DP it is important to understand the set of subproblems and the dependencies among them.
- The subproblem graph is a directed graph where the nodes are the subproblems and the edges represent the dependencies among them.
- Below is the graph for the IS example.



Interval Scheduling

- In a previous lecture we introduced the interval scheduling problem.
- We showed that the greedy strategy of selecting the interval with the smallest finishing time first does not work when the intervals have weights.
- The problem is formulated as follows:
- Let I be a set of intervals where each interval i starts at $e(i)$, ends at $f(i)$, and has value $v(i)$.
- Our goal is to find a subset of **non-overlapping** intervals $S \subseteq I$ such that $V = \sum_{i \in S} v(i)$ is maximum.

Weighted Interval Scheduling: DP solution

- Let $\mathcal{O}_n$ the optimal solution for the first n intervals and $opt(\mathcal{O}_n)$ be the value of the optimal solution.
- Our approach is similar to the independent set problem.
- We consider interval i_n and evaluate whether to include it in the solution or not.

Weighted Interval Scheduling: DP solution

- Let $\mathcal{O}_n$ the optimal solution for the first n intervals and $opt(\mathcal{O}_n)$ be the value of the optimal solution.
- Our approach is similar to the independent set problem.
- We consider interval i_n and evaluate whether to include it in the solution or not.
- If we include it in $\mathcal{O}_n$, how would it relate to the subproblems?

Weighted Interval Scheduling: DP solution

- Let $\mathcal{O}_n$ the optimal solution for the first n intervals and $opt(\mathcal{O}_n)$ be the value of the optimal solution.
- Our approach is similar to the independent set problem.
- We consider interval i_n and evaluate whether to include it in the solution or not.
- If we include it in $\mathcal{O}_n$, how would it relate to the subproblems?
- If we exclude it from $\mathcal{O}_n$, how would it relate to the subproblems?

Weighted Interval Scheduling: DP solution

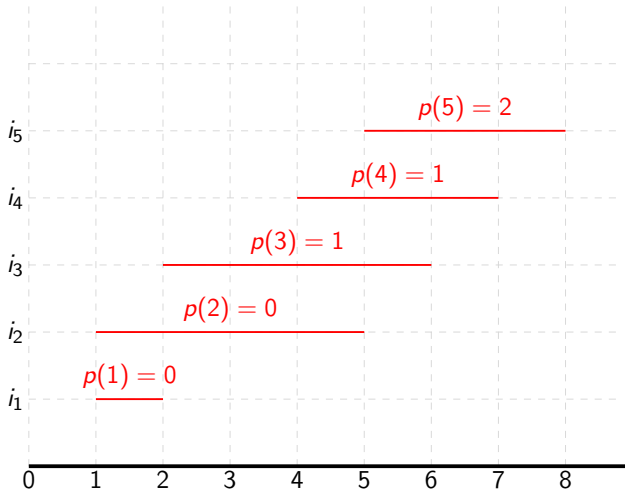
- Let $\mathcal{O}_n$ the optimal solution for the first n intervals and $opt(\mathcal{O}_n)$ be the value of the optimal solution.
- Our approach is similar to the independent set problem.
- We consider interval i_n and evaluate whether to include it in the solution or not.
- If we include it in $\mathcal{O}_n$, how would it relate to the subproblems?
- If we exclude it from $\mathcal{O}_n$, how would it relate to the subproblems?
- Let $overlap(i_k)$ be the set of intervals overlapping with interval k and denote the remaining set of intervals by $I_k = I \setminus overlap(i_k)$

$$opt(\mathcal{O}_n) = \max \begin{cases} v_n + opt(I_n) & \text{if } i_n \in \mathcal{O}_n \\ opt(\mathcal{O}_{n-1}) & \text{if } i_n \notin \mathcal{O}_n \end{cases}$$

Weighted Interval Scheduling: DP solution

- Computing $overlap(i_k)$ implies that we have to do $n - 1$ overlap tests which leads to an $\Omega(n^2)$ algorithm.
- Instead, first we sort the intervals by ending time (this is how greedy works in the "plain" version)
- We maintain an array p where $p[j] = k$ is the largest index $k < j$ such that intervals i_k and i_j do not overlap.
- Because the intervals are sorted by ending time then no interval with index $\leq p[j]$ can overlap with i_j .
- After the intervals are sorted we can compute $p[j]$ in $O(\log n)$ time for a total of $O(n \log n)$ time.

Example of p array



Using the P array

- Using the p array, what intervals are excluded from the optimal solution?

Using the P array

- When the intervals are sorted and we use the p array, the recurrence simplifies to
 - $i_n \in \mathcal{O}_n$. In this case all the intervals $i_{p(n)+1}, \dots, i_{n-1}$ overlap with i_n and therefore cannot be in $\mathcal{O}_n$
 - $i_n \notin \mathcal{O}_n$. In this case $\mathcal{O}_n = \mathcal{O}_{n-1}$ since we know that $i_n \notin \mathcal{O}_n$
- This means that we compute both possibilities and the larger of the two values is chosen as the optimal solution

$$opt(n) = \max \begin{cases} v_n + opt(p(n)) & \text{if } i_n \in \mathcal{O}_n \\ opt(n-1) & \text{if } i_n \notin \mathcal{O}_n \end{cases}$$

- The above is a recursive equation that will allow us to compute the optimal value.

Bottom up solution

Algorithm 5: Bottom up solution for weighted interval scheduling

input: values of sorted intervals v and p array

Result: Returns array opt the optimal values for all subproblems

function $OPT(v, p)$

$opt[0] = 0$

$opt[1] = v[1]$

for $i = 2$ **to** n **do**

$opt[i] = \max(opt[i - 1], v[i] + opt[p[i]])$

return opt

- For simplicity items are numbered starting from 1, i.e. $v[0]$ and $p[0]$ are unused.

Complexity

- Clearly the complexity of the above algorithm is $\Theta(n)$.
- We also need to sort the intervals by ending time which is (say quicksort) $O(n \log n)$, and computing the p array is also $O(n \log n)$ (binary search for each $p[i]$).
- Therefore the overall complexity is $O(n \log n)$

Computing the solution

Recovering the optimal interval list is done as before by going backwards over the recurrence relation

Algorithm 6: Computing optimal list of intervals for WIS

input: values of the array opt and p array

Result: Returns list of intervals leading to optimal solution

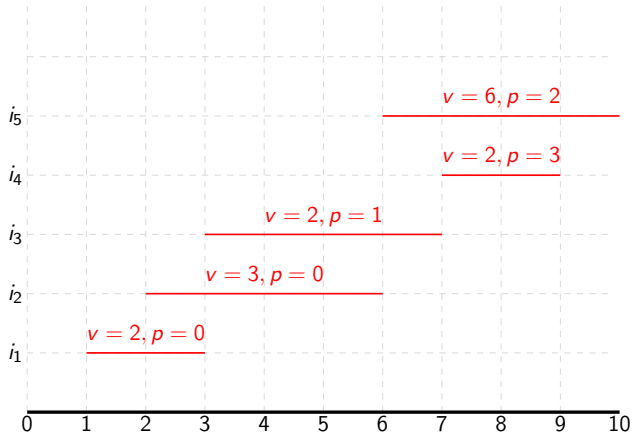
function OPT-SOL(opt, p)

```

     $i \leftarrow n$ 
    while  $i > 0$  do
        if  $opt[i] > opt[i - 1]$  then
             $S \leftarrow S \cup \{i\}$ 
             $i \leftarrow p[i]$ 
        else
             $i \leftarrow i - 1$ 
    return  $S$ 

```

Example



Example

$$opt[i] = \max(opt[i - 1], v[i] + opt[p[i]])$$

$$opt[1] = 2$$

<i>i</i>	0	1	2	3	4	5
<i>v</i>	0	2	3	2	2	6
<i>p</i>	0	0	0	1	3	2
<i>opt</i>	0	2				

Example

$$\text{opt}[i] = \max(\text{opt}[i - 1], v[i] + \text{opt}[p[i]])$$

$$\begin{aligned}\text{opt}[2] &= \max(\text{opt}[1], 3 + \text{opt}[0]) \\ &= \max(2, 3 + 0)\end{aligned}$$

i	0	1	2	3	4	5
v	0	2	3	2	2	6
p	0	0	0	1	3	2
opt	0	2	3			

Example

$$opt[i] = \max(opt[i - 1], v[i] + opt[p[i]])$$

$$\begin{aligned} opt[3] &= \max(opt[2], 2 + opt[1]) \\ &= \max(3, 2 + 2) \end{aligned}$$

<i>i</i>	0	1	2	3	4	5
<i>v</i>	0	2	3	2	2	6
<i>p</i>	0	0	0	1	3	2
<i>opt</i>	0	2	3	4		

Example

$$\text{opt}[i] = \max(\text{opt}[i - 1], v[i] + \text{opt}[p[i]])$$

$$\begin{aligned}\text{opt}[4] &= \max(\text{opt}[3], 2 + \text{opt}[3]) \\ &= \max(4, 2 + 4)\end{aligned}$$

<i>i</i>	0	1	2	3	4	5
<i>v</i>	0	2	3	2	2	6
<i>p</i>	0	0	0	1	3	2
<i>opt</i>	0	2	3	4	6	

Example

$$opt[i] = \max(opt[i - 1], v[i] + opt[p[i]])$$

$$\begin{aligned} opt[5] &= \max(opt[4], 6 + opt[2]) \\ &= \max(6, 6 + 3) \end{aligned}$$

<i>i</i>	0	1	2	3	4	5
<i>v</i>	0	2	3	2	2	6
<i>p</i>	0	0	0	1	3	2
<i>opt</i>	0	2	3	4	6	9

Example: Recovering the list of intervals

$$S = \{5, 2\}, p[5] = 2$$

<i>i</i>	0	1	2	3	4	5
<i>v</i>	0	2	3	2	2	6
<i>p</i>	0	0	0	1	3	2
<i>opt</i>	0	2	3	4	6	9

Example: Recovering the list of intervals

$$S = \{5, 2\}, p[2] = 0$$

<i>i</i>	0	1	2	3	4	5
<i>v</i>	0	2	3	2	2	6
<i>p</i>	0	0	0	1	3	2
<i>opt</i>	0	2	3	4	6	9

Extra: Time complexity of recursive Fibonacci

Let $T(n)$ be the cost of recursive Fibonacci on input n . The complexity obeys the recurrence

$$T(n) = T(n-1) + T(n-2) + c$$

Where c is a constant. Let $t(n) = T(n) - c$, then

$$t(n) = t(n-1) + t(n-2)$$

Let $t(n) = x^n$ thus

$$x^n = x^{n-1} + x^{n-2}$$

$$x^2 - x - 1 = 0$$

$x = \frac{1 \pm \sqrt{5}}{2}$ where $\phi = \frac{1 + \sqrt{5}}{2}$ is the Golden ratio.

References

- The linear independent set algorithm using DP is covered in sections 16.1,2,3 of the Algorithms Illuminated [book](#)
- A Python implementation of the independent set problem can be found [here](#), for WIS [here](#).